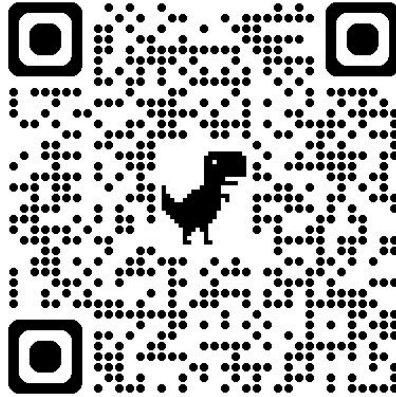


Vote for Your Favorite Paper



<https://app.sli.do/event/opSdyNn6dFBd2YDQyYoB3G>

Or visit <https://slido.com> and enter code: 3832076

Inference Basics

September 28, 2026

Kevin Xue

(Slides Adapted from Jae-Wong Chung)

What is Inference?

- Recall: GPT-like LLMs treat all tasks as text completion
- Inference: use a trained LLM to generate the output tokens
- LLM inference serving: a deployed LLM inference service for many, many users' inputs

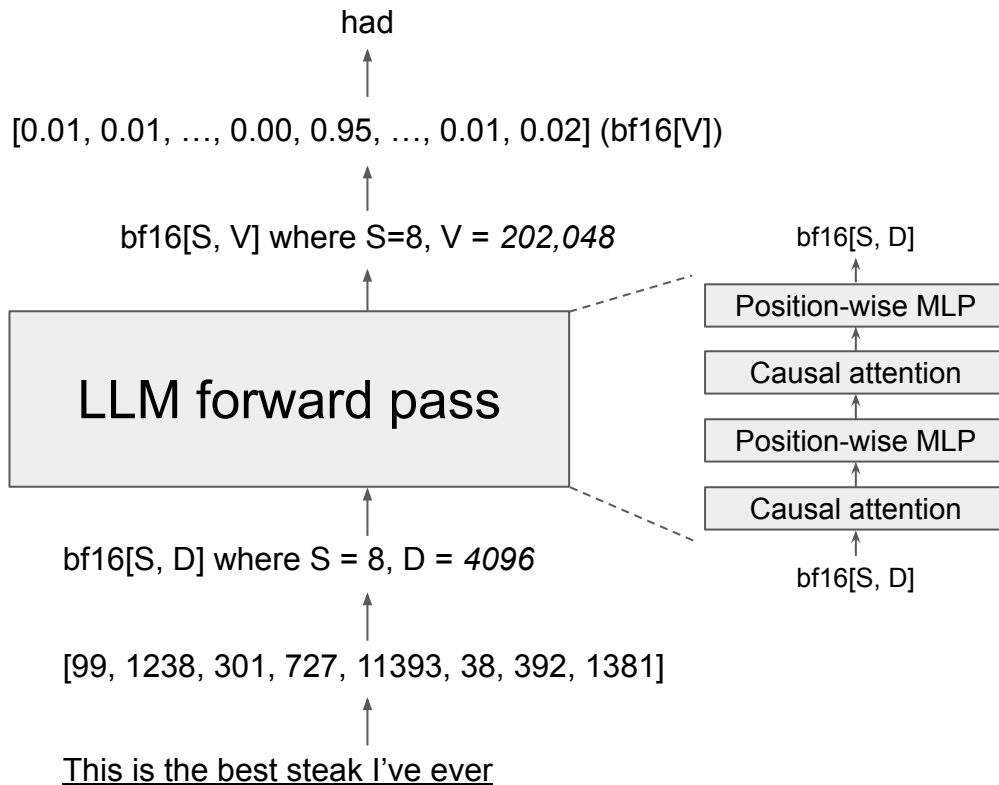
Today

- The actual computation performed during LLM inference
- Characteristics of an LLM inference workload
- Basic system optimizations of LLM inference serving

Today

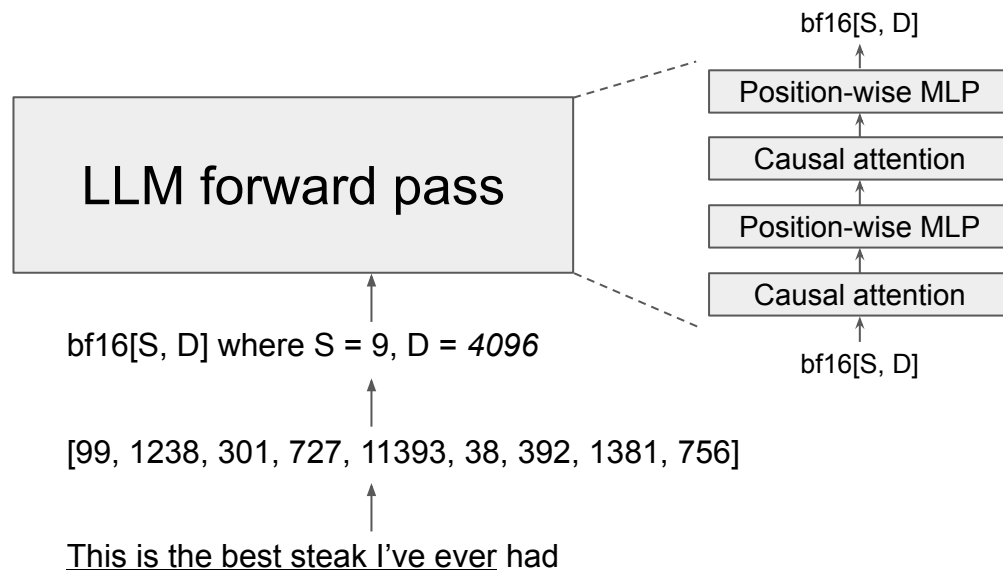
- The actual computation performed during LLM inference
 - How is the prediction of an output token actually computed?
 - What is KV cache and why do we need it?
 - The concept of prefill and decode
- Characteristics of an LLM inference workload
- Basic system optimizations of LLM inference serving

LLM Inference



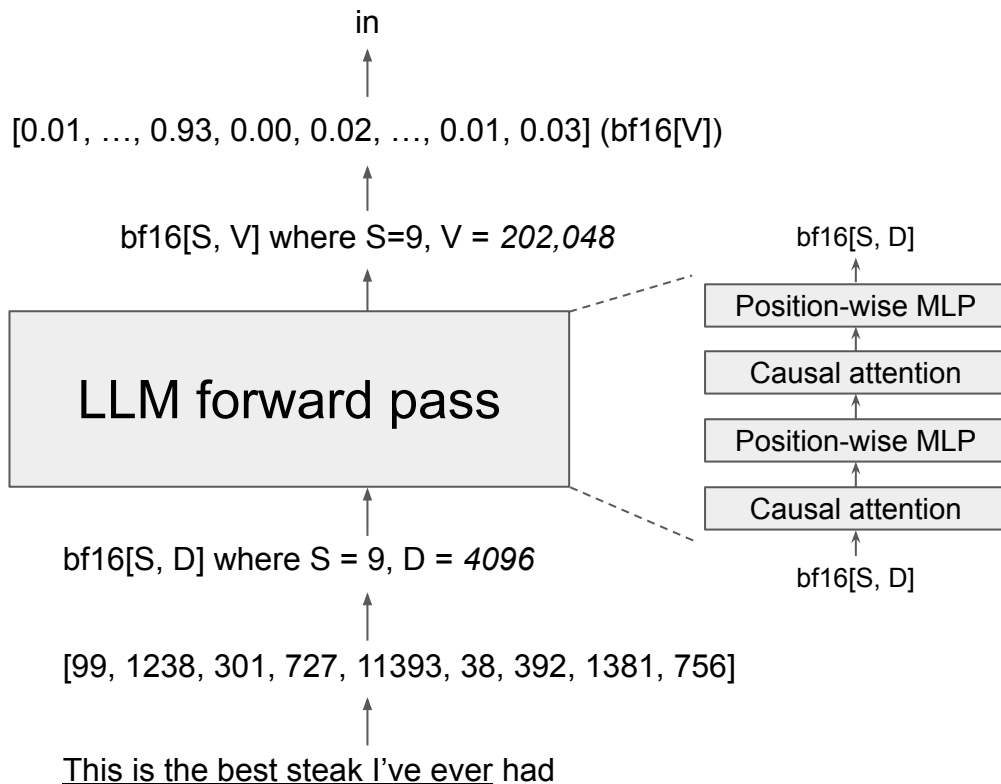
S = Sequence length
D = Dimension of embeddings
V = Vocabulary size

LLM Inference



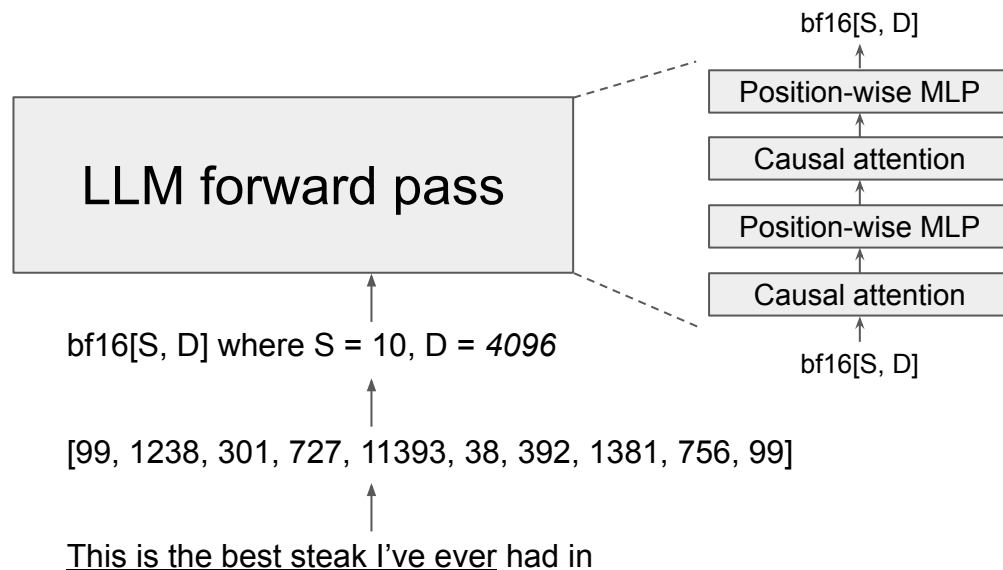
S = Sequence length
D = Dimension of embeddings
V = Vocabulary size

LLM Inference



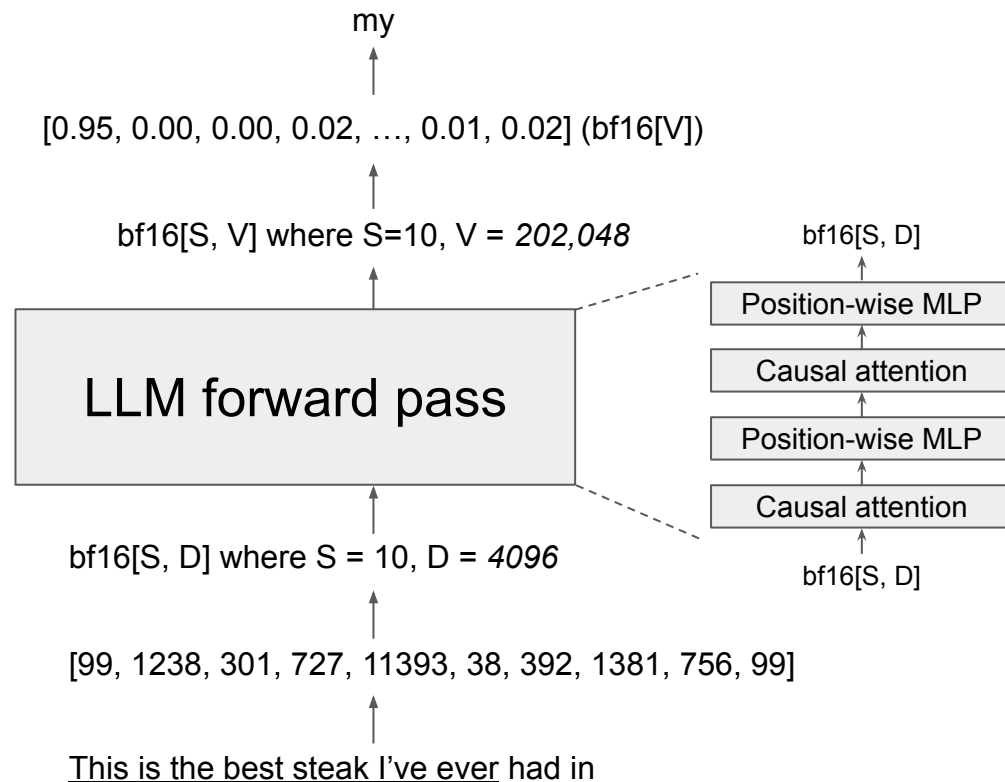
S = Sequence length
D = Dimension of embeddings
V = Vocabulary size

LLM Inference



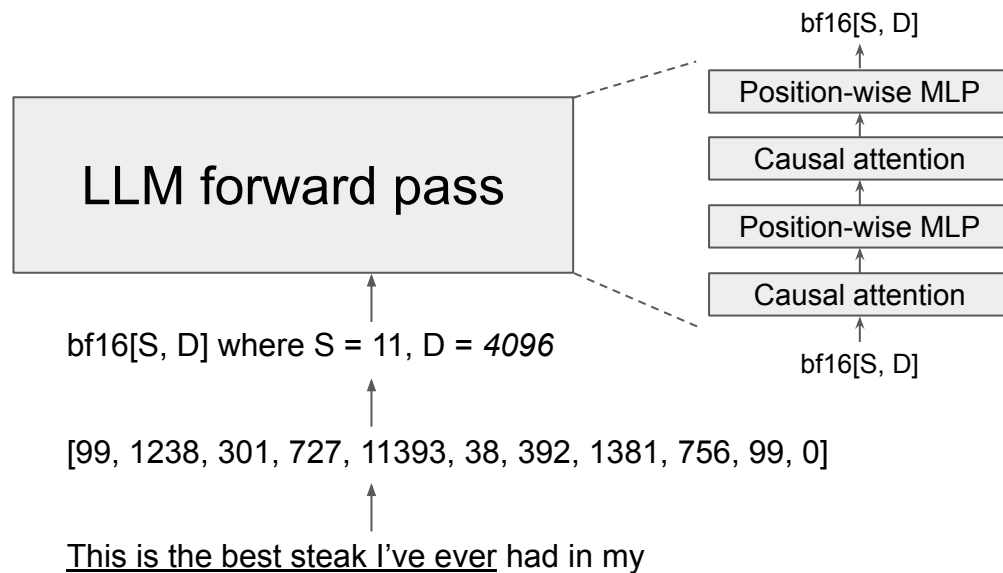
S = Sequence length
D = Dimension of embeddings
V = Vocabulary size

LLM Inference



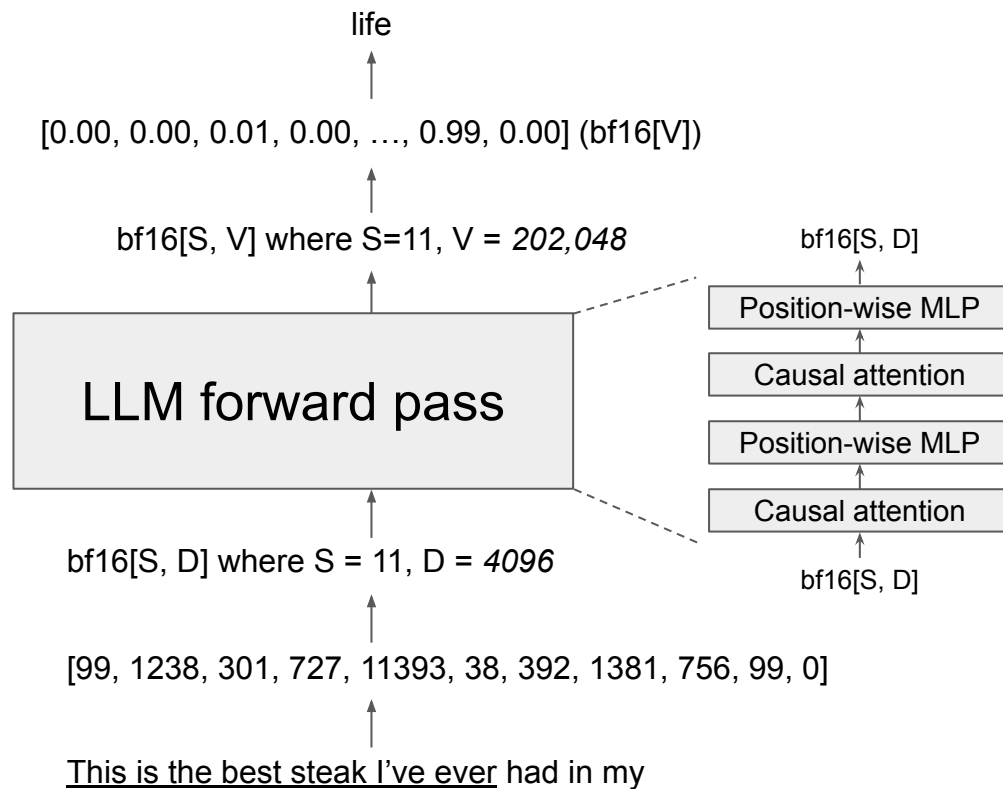
S = Sequence length
D = Dimension of embeddings
V = Vocabulary size

LLM Inference



S = Sequence length
D = Dimension of embeddings
V = Vocabulary size

LLM Inference

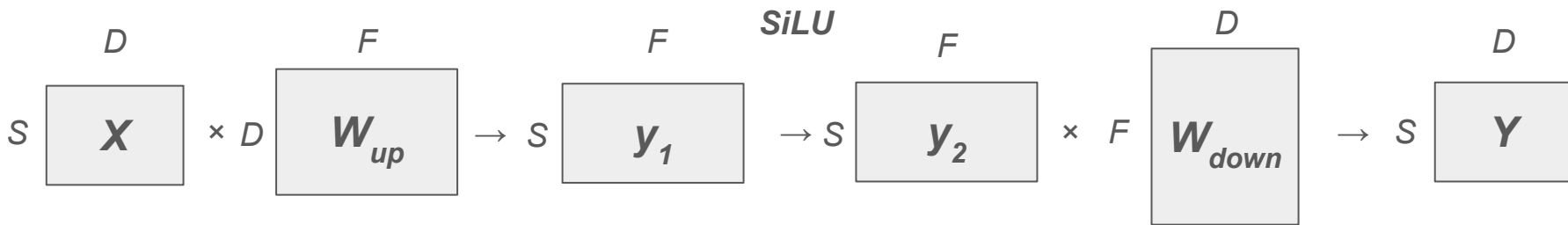


S = Sequence length
D = Dimension of embeddings
V = Vocabulary size

Prefill and Decode

- Prefill: process the prompt and generate the first output token
- Decode: generate the rest of the output tokens auto-regressively

Prompt Processing: MLP



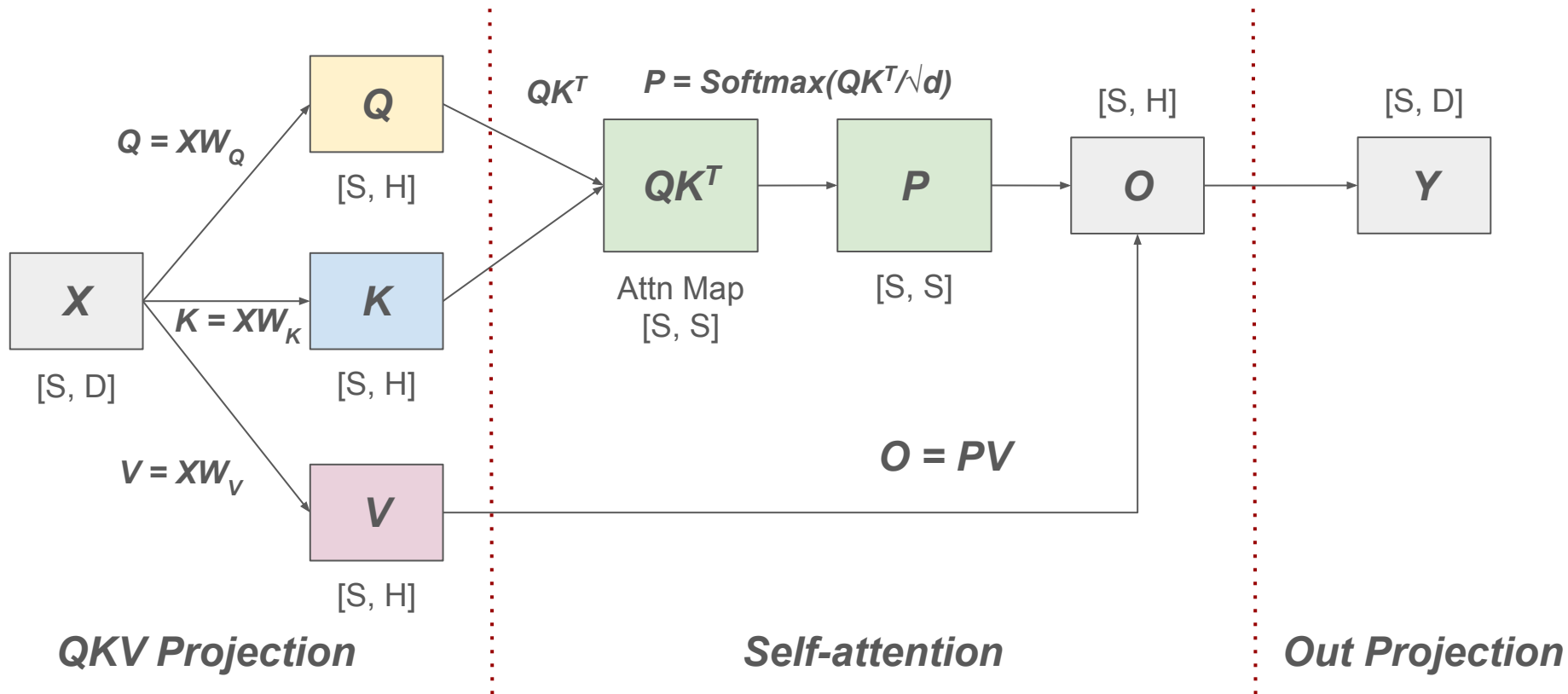
$$y = SiLU(XW_{up})W_{down}$$

S = Sequence length of the prompt

D = Dimension of embeddings

F = The hidden size of MLP intermediate

Prompt Processing: Attention



Generating the Second Output Token

Q: Do we need to redo the computation for the prompt?



<https://app.sli.do/event/opSdyNn6dFBd2YDQyYoB3G>

Or visit <https://slido.com> and enter code: 3832076

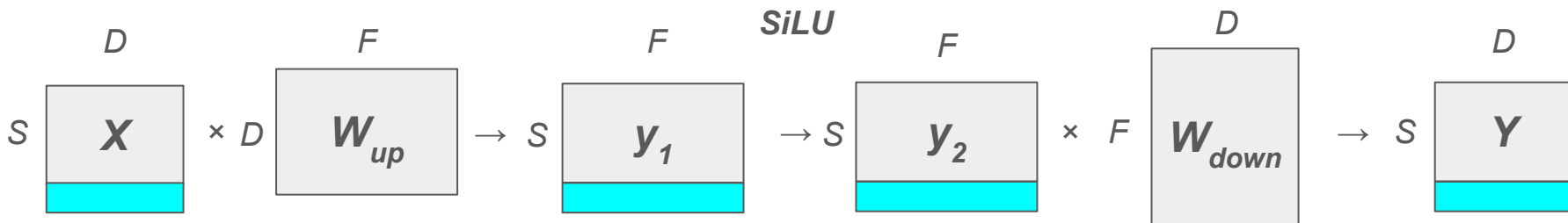
Generating the Second Output Token

Q: Do we need to redo the computation for the prompt?

A: Needed for both MLP and attention!

- Attention
 - The new input sequence length is $S+1$
 - We want to calculate the relationships of $(0, S+1), (1, S+1), \dots, (S, S+1)$
 - This needs the projection of all previous tokens!
- MLP
 - It has to compute the whole output matrix for attention to perform QKV projection, so prompt recomputation is still needed!

MLP is Position-wise



$$y = \text{SiLU}(XW_{up})W_{down}$$

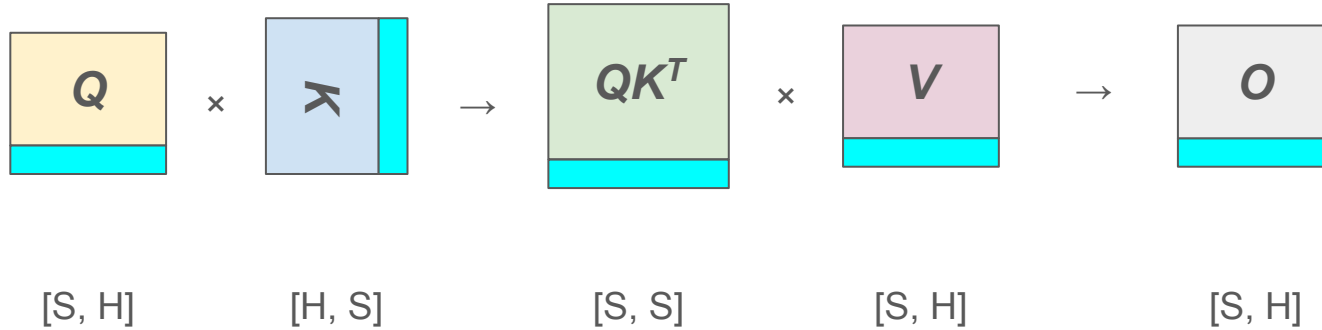
*The output of position X depends only on the input of position X and **not on the input of other positions**.*

S = Sequence length of the prompt

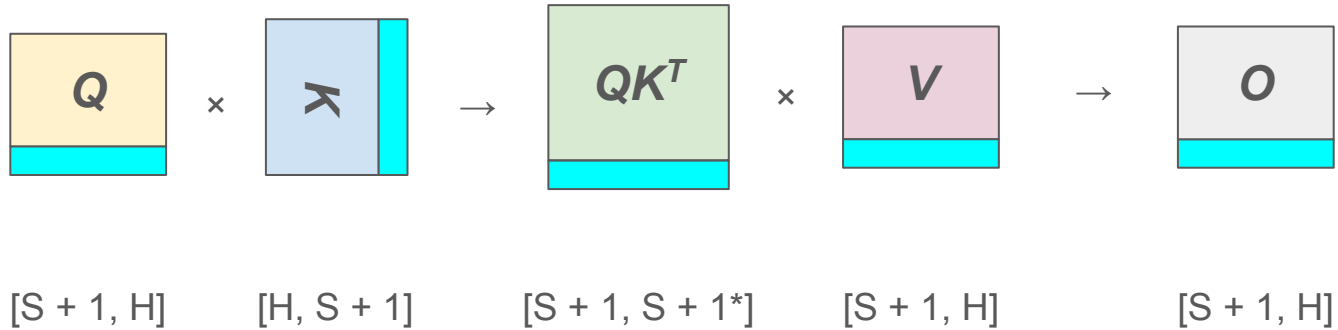
D = Dimension of embeddings

F = The hidden size of MLP intermediate

Attention Needs a History of All Tokens

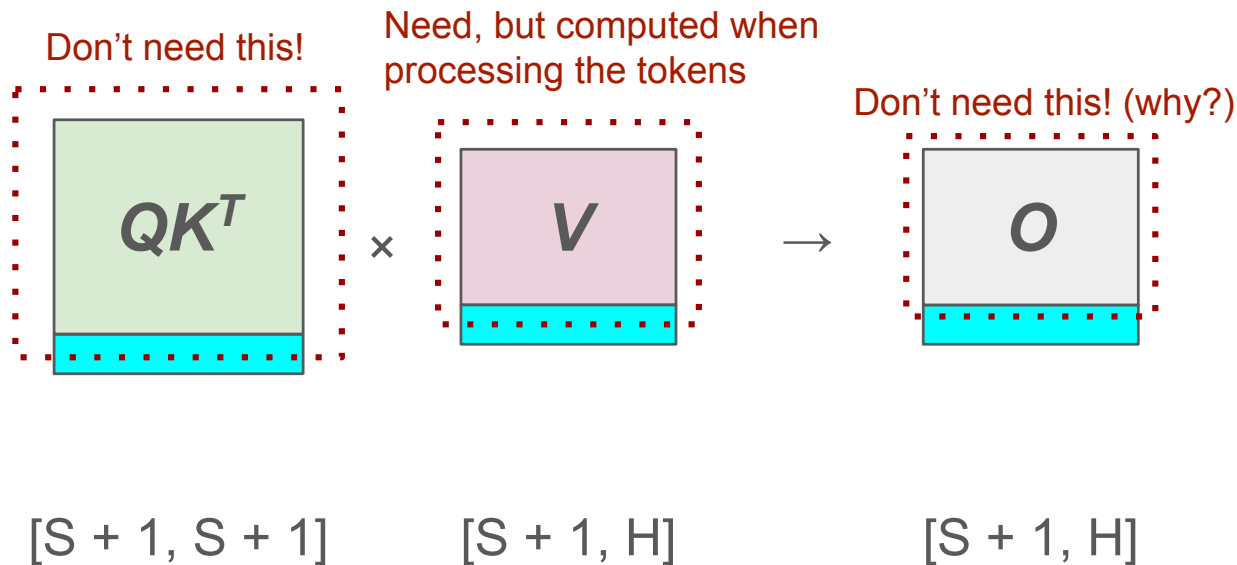


Attention Needs a History of All Tokens

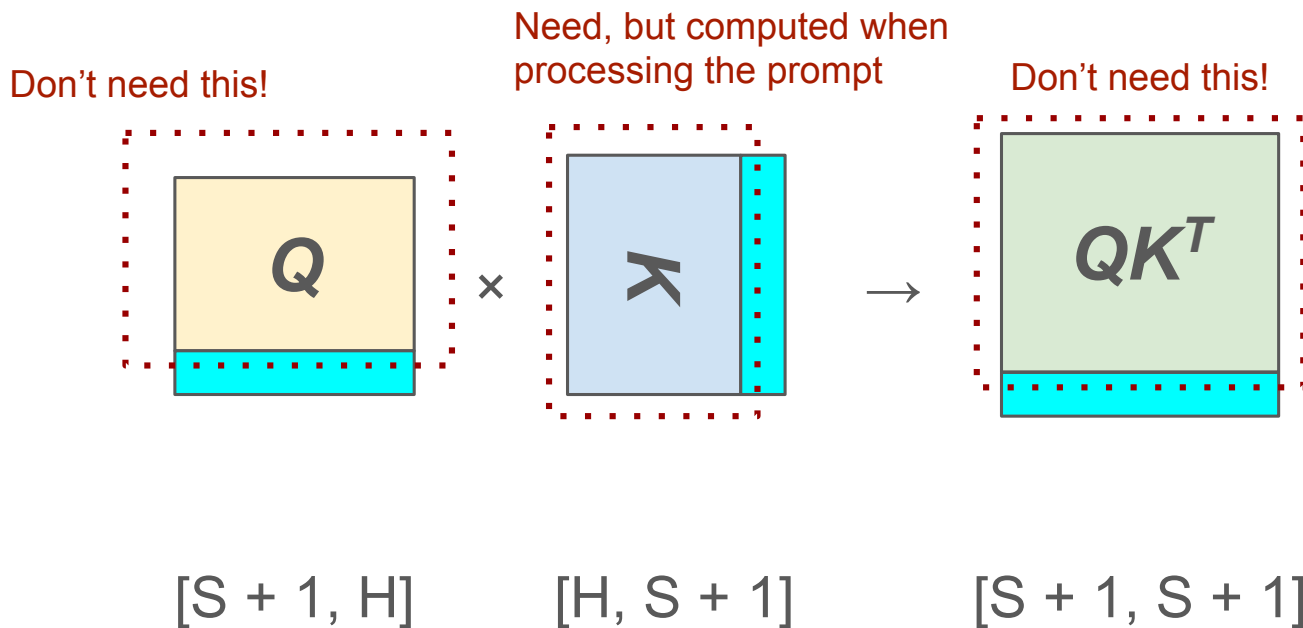


*: The additional column is omitted here because attention is causal

Caching V for Output



Caching K for Attention Map

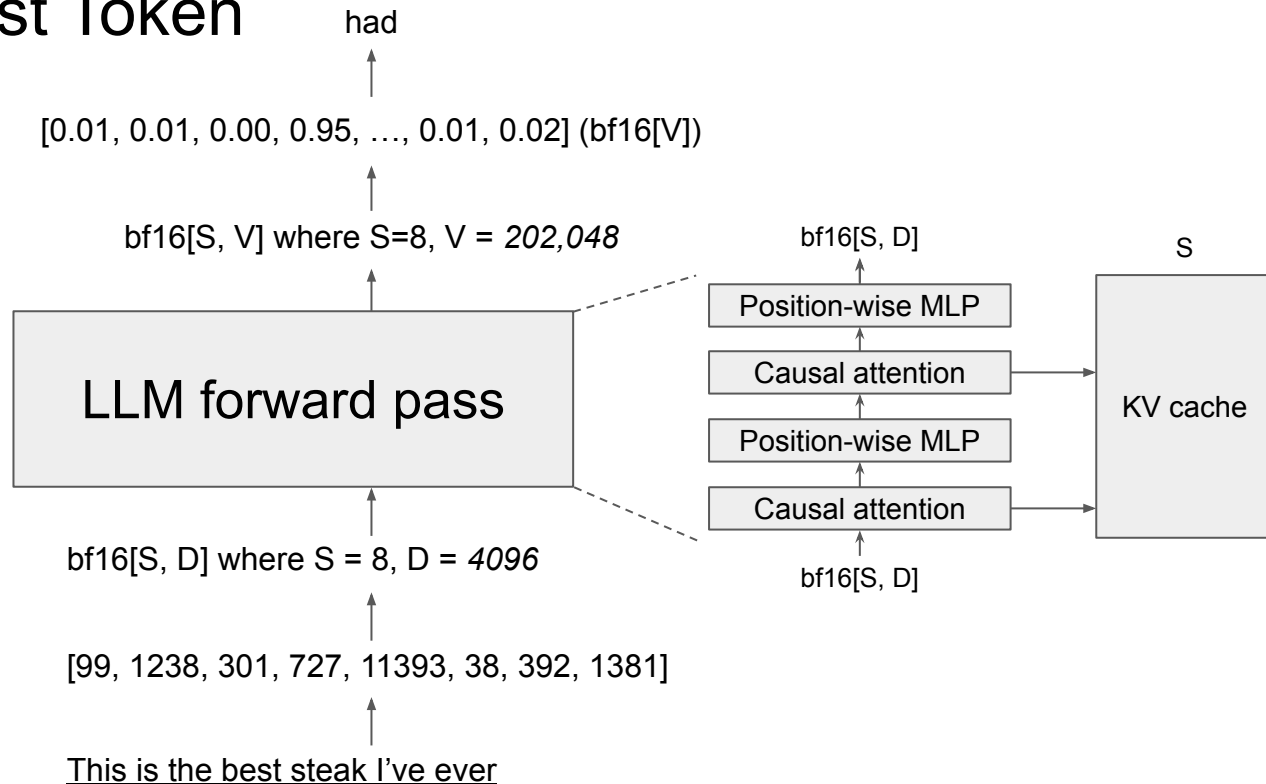


KV Cache

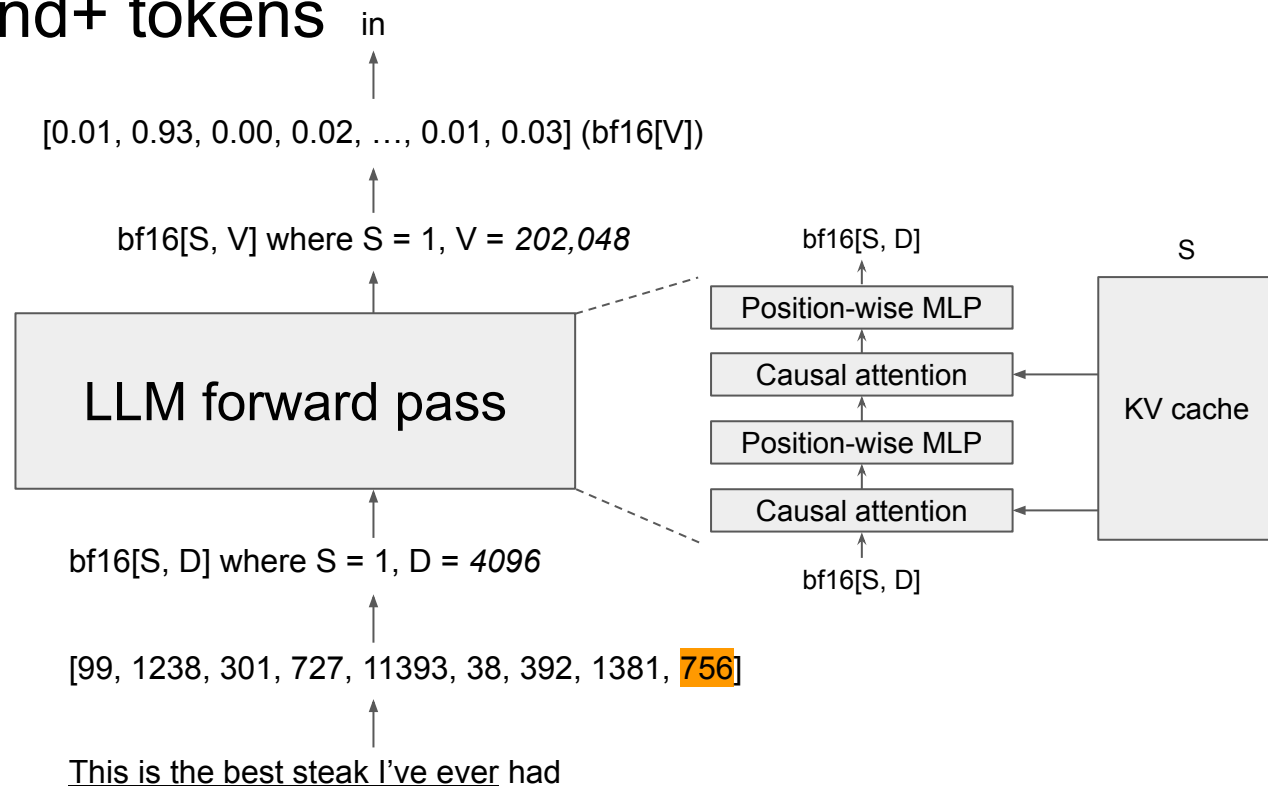
In order to get an output of O of size $[1, H]$

- We **do not need** the last $[S, H]$ history for the **Q matrix**
- We **do need** the last $[S, H]$ history for the **K matrix and V matrix**
 - Which people call KV cache
- Idea: **Prefill** the KV cache (size: $[2, S, H]$) when doing prompt processing
 - When generating each new token, we add a new $[1, H]$ vector to the KV cache

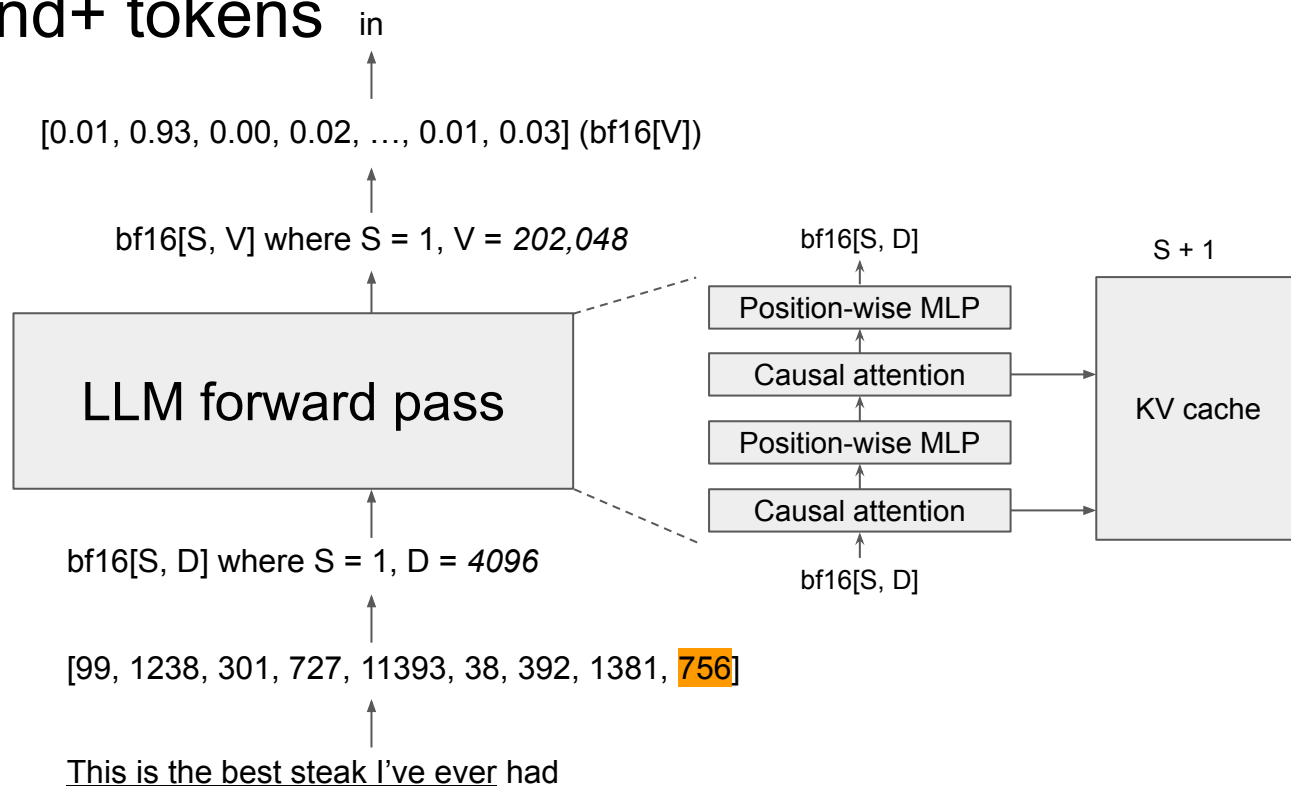
Prefill: The First Token



Decode: Second+ tokens in



Decode: Second+ tokens



Agenda

- The actual computation performed during LLM inference
- Characteristics of an LLM inference workload
- Basic system optimizations of LLM inference serving

KV Cache Sizes

- Characteristics
 - Constant size for each token in the context (input or output)
 - So, for each output token generated, KV cache size grows linearly
- Size calculation for a single token
 - $(\# \text{ Transformer layers}) \times (\# \text{ KV heads}) \times 2 \text{ (K and V)} \times (\text{head output dimension}) \times 2 \text{ bytes}$

Model	1 token	4096 tokens
Llama 3.1 8B	$32 \times 8 \times 2 \times 128 \times 2 \text{ bytes} = 128 \text{ KiB}$	0.50 GiB
Llama 3.1 70B	$80 \times 8 \times 2 \times 128 \times 2 \text{ bytes} = 320 \text{ KiB}$	1.25 GiB
Llama 3.1 405B	$126 \times 8 \times 2 \times 128 \times 2 \text{ bytes} = 504 \text{ KiB}$	1.97 GiB

Optimization Metrics

- Prefill latency
 - **Time To First Token (TTFT)**
 - TTFT + queuing delay is the user-experienced wait time
- Decode latency
 - **Time Per Output Token (TPOT)**: Average decode latency for all output tokens
 - **Inter-Token Latency (ITL)**: Latencies between each output token
 - Long decode latency leads to a laggy chat experience and will annoy users
 - It also doesn't have to be infinitely fast if the LLM is user-facing
- Throughput
 - Request throughput (reqs/s)
 - Token generation throughput (tokens/s)

A Potential Optimization: Batching

- Training had batch size; what about for inference?
 - Running more than one requests together
- LLMs are pretty weird
 - Very large in size
 - Prefill and decode
- Questions
 - Is batching a no-brainer optimization, or does it only sometimes make sense?
 - How should we *reason* about batching for LLMs on modern GPUs?

Matrix Multiplication

- It's everywhere in Transformers and takes up most of the compute
 - Some in attention, and the MLP is basically two *big* matmuls
- Focus: MLP
 - Batch size (B) is the total number of tokens being forwarded through the MLP
 - Say we batched together three requests with input sequence length: 3, 4, and 5

Input	bf16[B, D]
Weight	bf16[D, F]
Output	bf16[B, F]

Matrix Multiplication

- It's everywhere in Transformers and takes up most of the compute
 - Some in attention, and the MLP is basically two *big* matmuls
- Focus: MLP
 - Batch size (B) is the total number of tokens being forwarded through the MLP
 - Say we batched together three requests with input sequence length: 3, 4, and 5
 - Prefill: $B = 12$ – total number of input tokens
 - Decode: $B = 3$ – number of tokens generated (== number of requests)

Input	bf16[B, D]
Weight	bf16[D, F]
Output	bf16[B, F]

Analyzing Matrix Multiplication

- Computation and memory accesses
 - $B = 128, D = 4096, F = 16384$

Input	bf16[B, D]
Weight	bf16[D, F]
Output	bf16[B, F]

	Computation	Memory
Matmul	$2BDF$ $= 16 \times 10^9 \text{ FLOPs}$	$2BD + 2DF + 2BF$ $= 0.13 \times 10^9 \text{ Bytes}$

Analyzing Matrix Multiplication

- Computation and memory accesses
 - $B = 128, D = 4096, F = 16384$

Input	bf16[B, D]
Weight	bf16[D, F]
Output	bf16[B, F]

	Computation	Memory
Matmul	$2BDF$ $= 16 \times 10^9 \text{ FLOPs}$	$2BD + 2DF + 2BF$ $= 0.13 \times 10^9 \text{ Bytes}$
H100 spec	$1979 \text{ TFLOPs/s} / 2$ $= 989.5 \times 10^{12} \text{ FLOPs/s}$	3.35 TB/s $= 3.35 \times 10^{12} \text{ Bytes/s}$

Which one is bottlenecking our matmul?
Computation throughput, or memory throughput?

Arithmetic Intensity

- A single number that tells us which one we're bottlenecked by
- **Arithmetic Intensity**: Computation per byte of memory access
 - Amount of computation / amount of memory access (FLOPs/bytes)

	Computation	Memory
Matmul	2BDF $= 16 \times 10^9 \text{ FLOPs}$	$2\text{BD} + 2\text{DF} + 2\text{BF}$ $= 0.13 \times 10^9 \text{ Bytes}$
H100 spec	$1979 \text{ TFLOPs/s} / 2$ $= 989.5 \times 10^{12} \text{ FLOPs/s}$	3.35 TB/s $= 3.35 \times 10^{12} \text{ Bytes/s}$

Which one is bottlenecking our matmul?
Computation throughput, or memory throughput?

Arithmetic Intensity

- A single number that tells us which one we're bottlenecked by
- **Arithmetic Intensity**: Computation per byte of memory access
 - Amount of computation / amount of memory access (FLOPs/bytes)

	Computation	Memory	Computation per byte
Matmul	$2BDF$ $= 16 \times 10^9 \text{ FLOPs}$	$2BD + 2DF + 2BF$ $= 0.13 \times 10^9 \text{ Bytes}$	Exactly: 123 <u>Approximate with B</u> : 128
H100 spec	$1979 \text{ TFLOPs/s} / 2$ $= 989.5 \times 10^{12} \text{ FLOPs/s}$	3.35 TB/s $= 3.35 \times 10^{12} \text{ Bytes/s}$	Exactly: 295

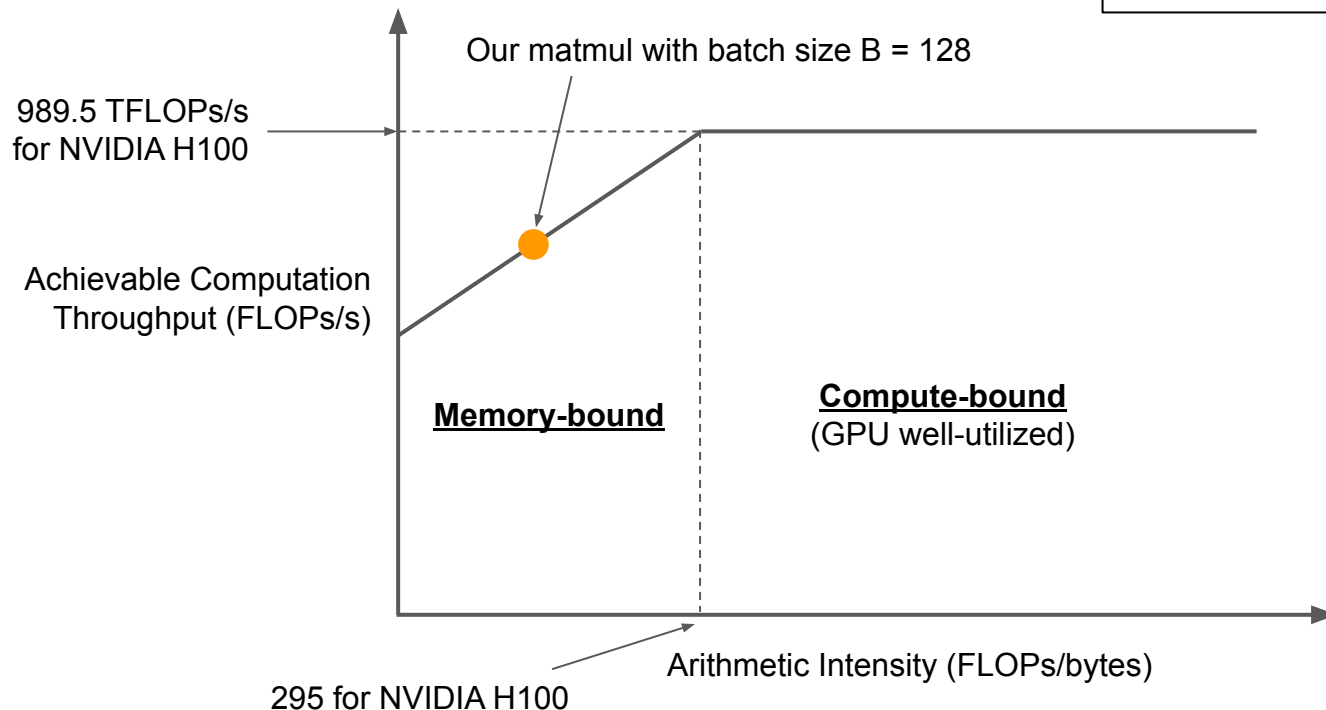
*It's **memory bound**.*

B = 128, D = 4096, F = 16384

The Roofline Plot

You get a different roofline for each

- Hardware
- Floating point precision
- Dense vs. sparse operation
- etc.



What All This Means for Prefill and Decode

- MLP

- Batch size (B) is the total number of tokens being forwarded through the MLP
- Say we batched together three requests with input sequence length: 3, 4, and 5
- Prefill: $B = 12$ – total number of input tokens
- Decode: $B = 3$ – number of tokens generated (== number of requests)

- Prefill and Decode

- **Prefill**: 295 tokens across input sequences is easy, so you're **nearly always compute-bound**
 - You actually don't have to run multiple *requests* together for prefill to be compute-bound

What All This Means for Prefill and Decode

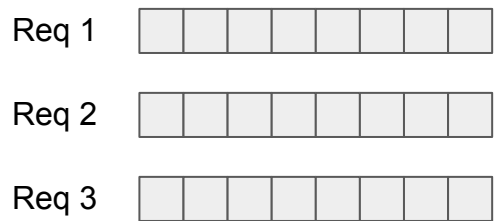
- Position-wise MLP
 - Batch size (B) is the total number of tokens being forwarded through the MLP
 - Say we batched together three requests with input sequence length: 3, 4, and 5
 - Prefill: $B = 12$ – total number of input tokens
 - Decode: $B = 3$ – number of tokens generated (== number of requests)
- Prefill and Decode
 - **Prefill**: 295 tokens across input sequences is easy, so you're **nearly always compute-bound**
 - You actually don't have to run multiple *requests* together for prefill to be compute-bound
 - **Decode**: Running 295 *requests* together is *not easy*, so you're **usually memory-bound**
 - We always want to **increase** the number of *requests* running together for decode

Agenda

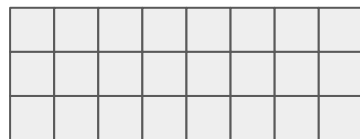
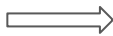
- The actual computation performed during LLM inference
- Characteristics of an LLM inference workload
- Basic system optimizations of LLM inference serving

Orca (OSDI '22): Efficient Batching for LLMs

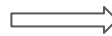
- If LLM inference requests
 - had the same number of input tokens, and
 - had the same number of output tokens (i.e., same number of decode iterations),
 - then batching is trivial



$\text{bf16}[S, D] \times N$



$\text{bf16}[N, S, D]$



Everyone runs the
same number of
iterations together

Orca (OSDI '22): Efficient Batching for LLMs

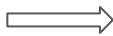
- If LLM inference requests
 - had the same number of input tokens, and
 - had the same number of output tokens (i.e., same number of decode iterations),
 - then batching is trivial

Req 1 

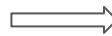
Req 2 

Req 3 

$\text{bf16}[S, D] \times N$



?



Everyone runs
different numbers of
iterations... *together*?

$\text{bf16}[N, S, D]$

Orca (OSDI '22): Efficient Batching for LLMs

- MLP

- MLP is position-wise anyway
- Just flatten all the tokens together into the batch dimension, and run forward
 - As if it is a single request with sequence length = $N * S$

Req 1 

Req 2 

Req 3 

bf16[S, D] x N



?



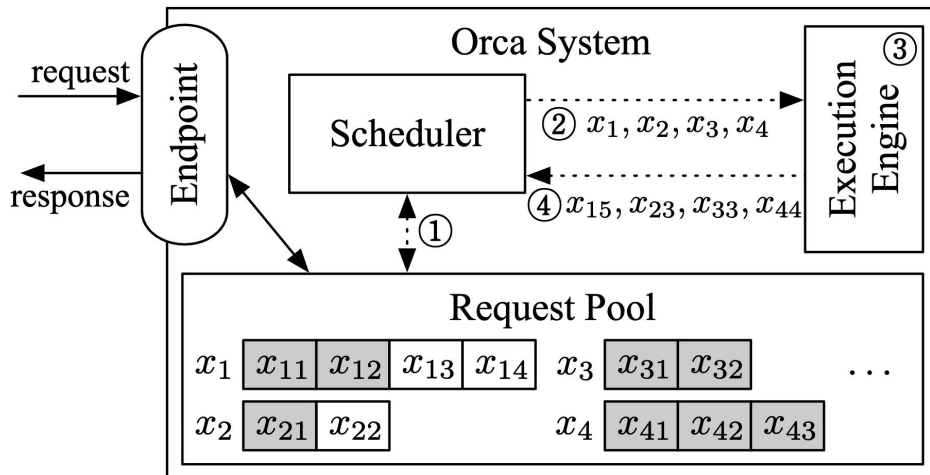
Everyone runs
different numbers of
iterations... *together*?

bf16[N, S, D]

Orca (OSDI '22): Efficient Batching for LLMs

- Scheduling benefits

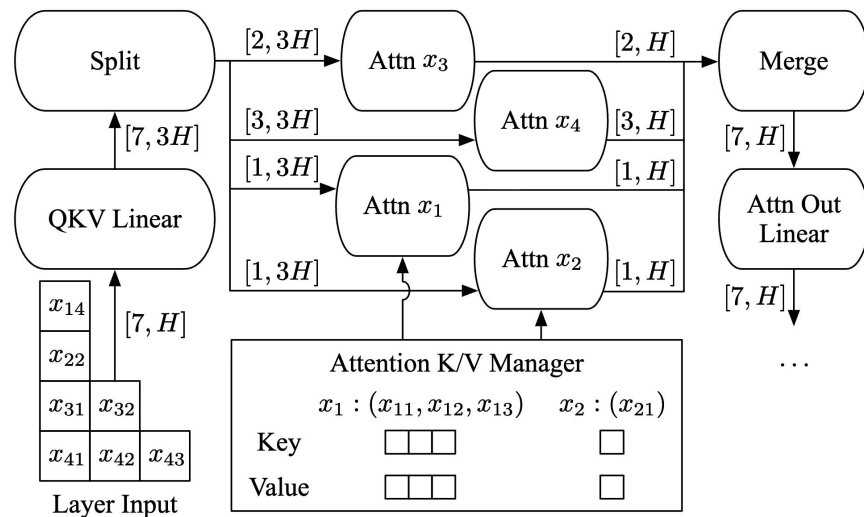
- Requests may enter and leave the system at iteration boundaries
- Run one iteration → Back to scheduler, finished ones leave the system
- New request → Enqueued into scheduler, can start processing when there's an empty slot



Orca (OSDI '22): Efficient Batching for LLMs

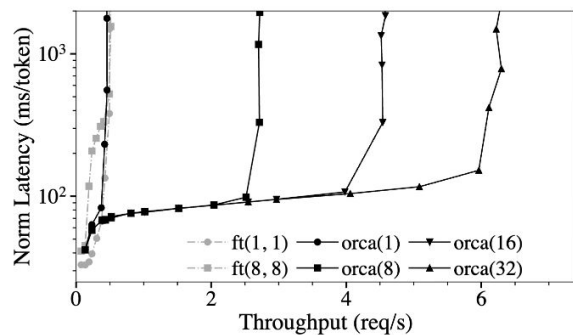
- Attention

- GPUs are parallel enough
- Just dispatch the attention operation of each sequence in parallel

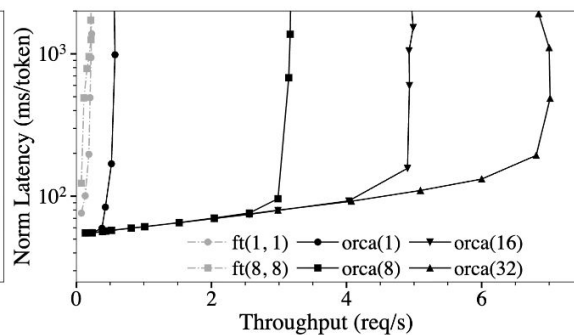


Orca (OSDI '22): Efficient Batching for LLMs

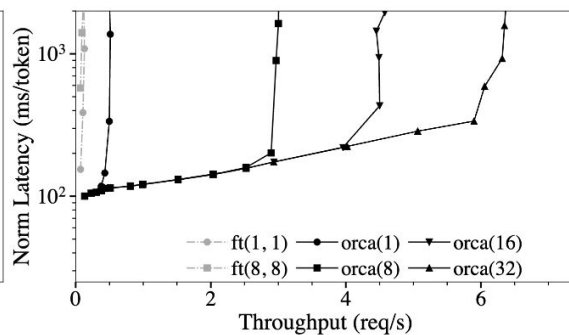
- Crushes baselines
 - NVIDIA A100-40GB GPUs
 - Batch size is in the order of *tens*



(a) 101B model, 8 GPU.



(b) 175B model, 16 GPUs.

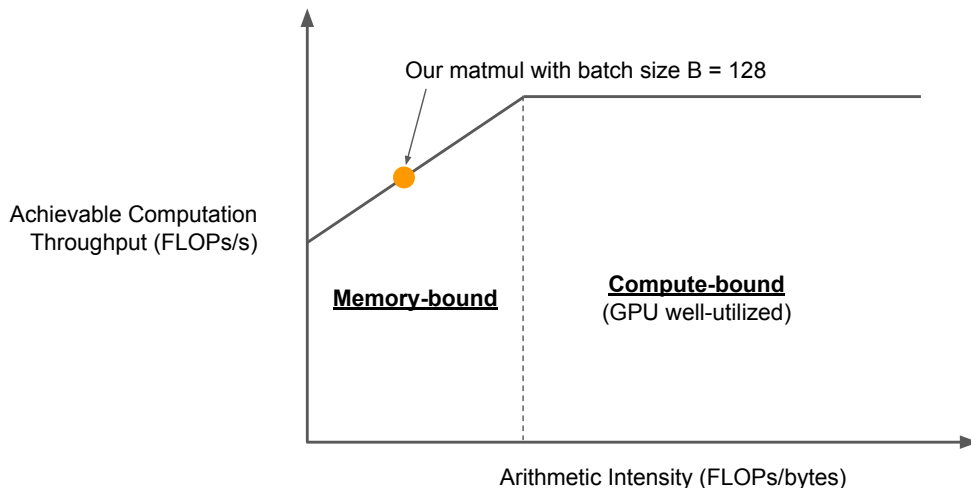


(c) 341B model, 32 GPUs.

Figure 10: Median end-to-end latency normalized by the number of generated tokens and throughput. Label “orca(*max_bs*)” represents results from ORCA with a max batch size of *max_bs*. Label “ft(*max_bs*, *mbs*)” represents results from FasterTransformer with a max batch size of *max_bs* and a microbatch size of *mbs*.

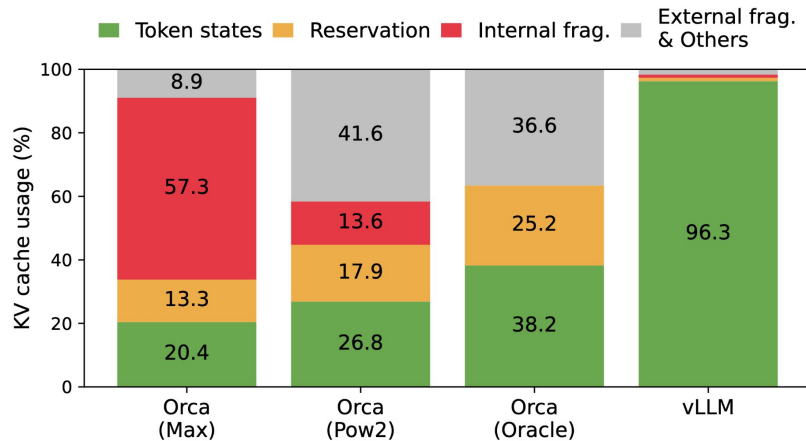
vLLM (SOSP '23): Memory is Key!

- Recall arithmetic intensity of matrix multiplication
 - We wanted a **larger batch size** for better hardware utilization & throughput
- Why can't we increase batch size very easily for Decode?
 - Each request needs to have its **KV cache** in GPU memory – hits **VRAM capacity**



vLLM (SOSP '23): Memory is Key!

- You don't know how much KV cache memory to allocate for a request
 - Because you don't know how many output tokens it'll generate
- KV cache memory pre-allocation leads to memory wastage
 - Max, Pow2 (Half-oracle), and Oracle: Policies for how much memory is pre-allocated
 - In all cases, memory is pre-allocated during the entire lifetime of the request

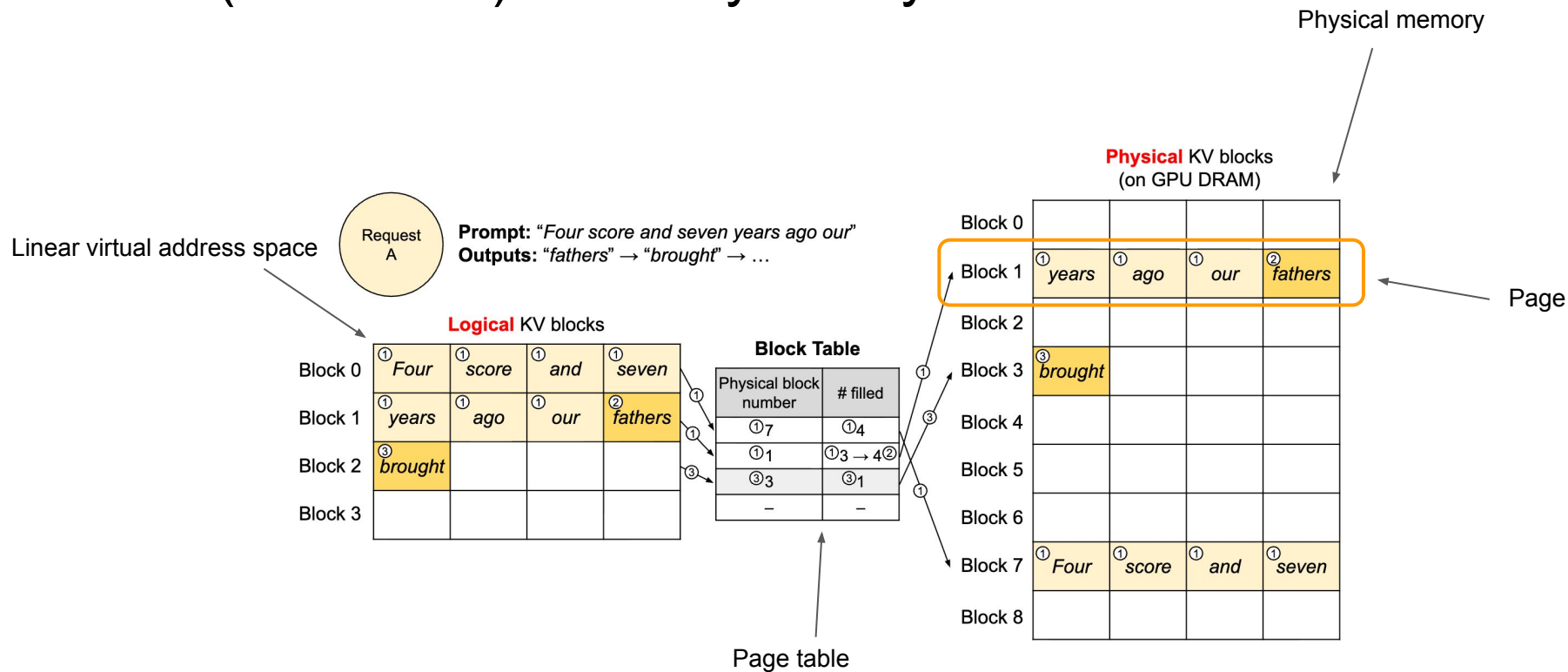


vLLM (SOSP '23): Memory is Key!

- This sounds familiar if you have taken 482
 - There are units of work, and they come and go
 - You don't know how much memory these units will consume (they can vary quite a bit)
 - When the unit goes away, you deallocate all memory associated with it
 - Like... *processes*

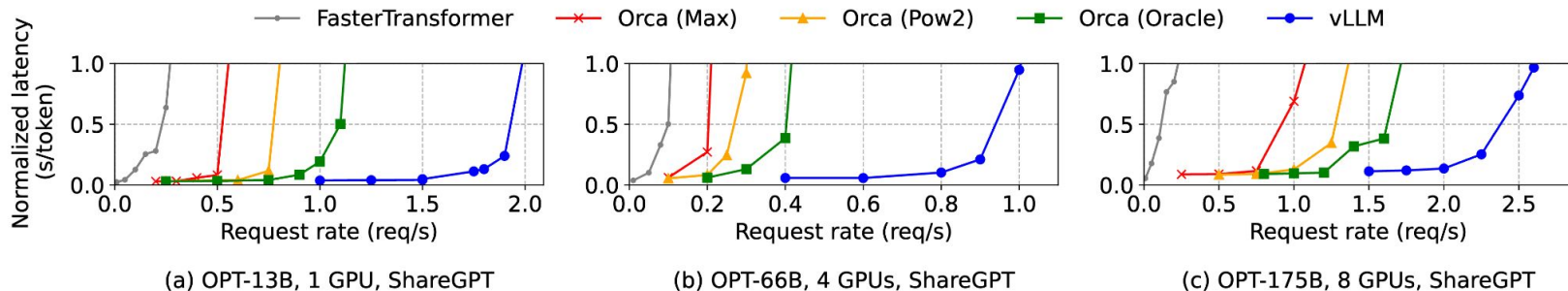
Virtual memory and pages

vLLM (SOSP '23): Memory is Key!



vLLM (SOSP '23): Memory is Key!

- Crushes baselines
 - NVIDIA A100-40GB and A100-80GB GPUs
 - For each system, uses the largest batch size that fits in memory



More Advanced Techniques On Wednesday

- If prefill and decode is so different in characteristics, why put them on the same GPU?
 - Prefill-Decode (P/D) disaggregation
- Is it possible we keep the GPU completely busy by fusing every operator in an LLM completely?
 - Mega kernel and mirage persistent kernel